Technical Asset Inventory

APJ Multi-Industry Predictive Maintenance Platform

This document provides a complete inventory of all technical assets, components, pipelines, tables, jobs, and deployment artifacts in the APJ Multi-Industry Predictive Maintenance platform. Use this as a reference for understanding the architecture, locating specific components, and planning modifications or extensions.

3 DLT Pipeline Layers	**12+** Delta Tables
8+ ML Models/Job	**25+** Python Modules
9	**5**

React Components

Scheduled Jobs

Table of Contents

1. Delta Live Tables (DLT) Pipelines
2. Delta Lake Tables Schema
3. Machine Learning Components
4. Application Backend (FastAPI)
5. Application Frontend (React)
6. Databricks Jobs & Workflows
7. Deployment & Configuration
8. Tools & Utilities
9. Integration Components

1. Delta Live Tables (DLT)

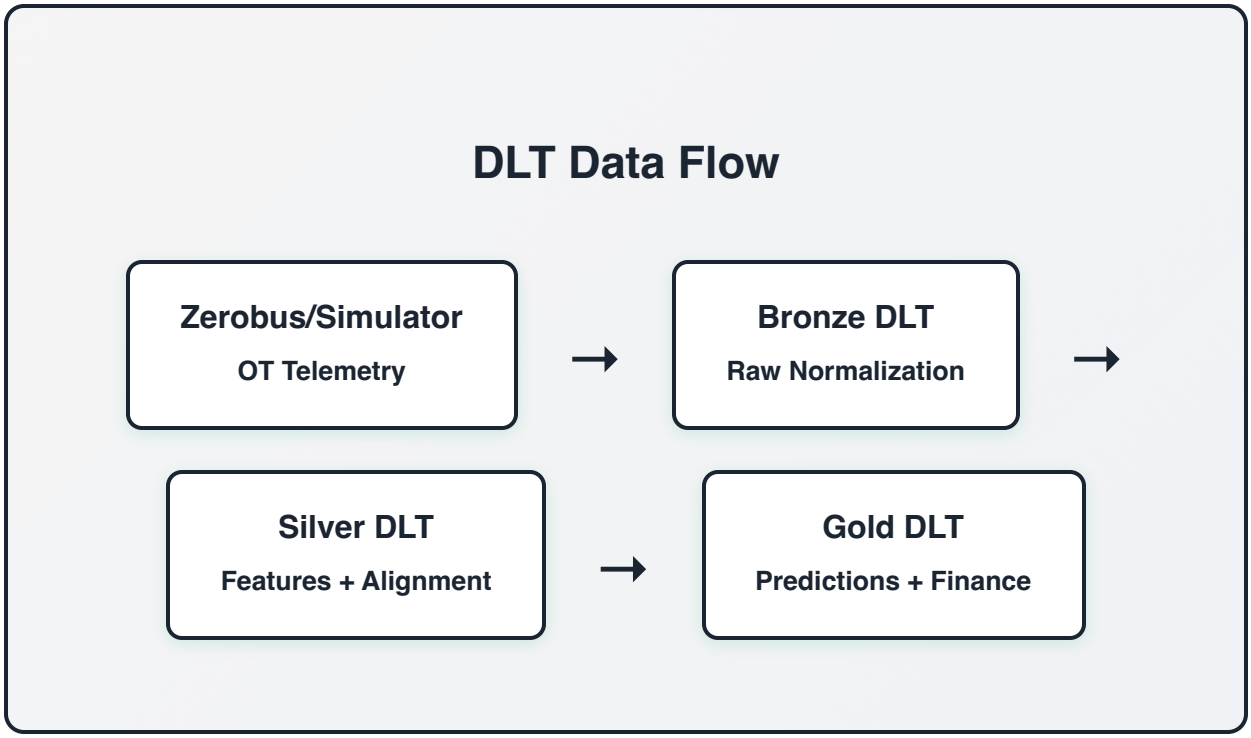
Pipelines

> The platform uses a three-tier medallion architecture implemented with Delta Live Tables for declarative, scalable ETL processing.

DLT Pipeline Components

Component	File Path	Layer	Description
Bronze Pipeline	`core/dlt/bronze.py`	Bronze	Raw telemetry ingestion from Zerobus and PI simulator. Normalizes OT/IoT streams into canonical schema. Applies data quality constraints.
Silver Pipeline	`core/dlt/silver.py`	Silver	Feature engineering with 15m/1h/30d windows. OT+PI time-alignment with provenance tracking. Quality-weighted aggregations.
Gold Pipeline	`core/dlt/gold.py`	Gold	ML-ready feature vectors. Prediction enrichment with financial context. Maintenance event

Component	File Path	Layer	Description
			materialization with ROI calculations.



Key DLT Functions

Bronze: Sensor Readings

Function: `@dlt.table sensor_readings()`

Streams from Zerobus landing table. Validates ISA-95 hierarchy (site/area/unit/equipment). Enforces quality codes (GOOD, UNCERTAIN, BAD).

Bronze: PI Simulated Tags

Function: `@dlt.table pi_simulated_tags()`

Ingests PI historian simulation data. Mirrors OT schema for alignment. Preserves source protocol metadata.

Silver: Sensor Features

Function: `@dlt.table sensor_features()`

Computes mean, stddev, slope, z-score over multiple windows. Quality-weighted aggregations. Excludes BAD quality readings.

Silver: OT-PI Alignment

Function: `@dlt.table ot_pi_aligned()`

Time-proximity join (30s window). Provenance tracking: BOTH, OT_ONLY, PI_ONLY, UNKNOWN. Delta computation for drift analysis.

Gold: PDM Predictions

Function: `@dlt.table pdm_predictions()`

Enriches ML model outputs with asset context. Links anomaly scores to RUL estimates. Identifies top contributing sensors.

Gold: Financial Impact

Function: `@dlt.table financial_impact_events()`

Merges predictions with maintenance windows, crew availability, parts inventory. Computes expected failure cost, avoided cost, total event cost.

2. Delta Lake Tables Schema

> All tables use Delta Lake format with ACID transactions, time travel, and change data feed enabled for operational integration.

Bronze Layer Tables

Table Name	Schema	Purpose	Key Columns
sensor_readings	Bronze	Primary telemetry stream from OT/IoT devices	equipment_id, tag_name, value, quality, timestamp
pravin_zerobus	Bronze	Zerobus connector landing table	site_id, equipment_id, tag_name, value, source_protocol
pi_simulated_tags	Bronze	PI historian simulation landing	equipment_id, tag_name, value, quality_code, timestamp

Table Name	Schema	Purpose	Key Columns
asset_metadata	Bronze	Asset master data and hierarchy	equipment_id, site_id, asset_type, industry, cost_per_unit

Silver Layer Tables

Table Name	Schema	Purpose	Key Columns
sensor_features	Silver	Engineered features for ML training and scoring	equipment_id, mean_15m, stddev_15m, slope_1h, zscore_30d
ot_pi_aligned	Silver	Time-aligned OT and PI data with provenance	ot_timestamp, ot_value, pi_timestamp, pi_value, data_source

Gold Layer Tables

Table Name	Schema	Purpose	Key Columns
pdm_predictions	Gold	ML model predictions with metadata	anomaly_score, anomaly_label, rul_hours, predicted_failure_date

Table Name	Schema	Purpose	Key Columns
financial_impact_events	Gold	Business-facing events with ROI calculations	maintenance_cost, expected_failure_cost, avoided_cost, total_event_cost

Lakebase OLTP Tables

Table Name	Schema	Purpose	Key Columns
parts_inventory	Lakebase	Real-time parts availability for maintenance planning	part_number, quantity, unit_cost, currency, reorder_point
work_orders	Lakebase	Maintenance work order management	work_order_id, equipment_id, priority, status, scheduled_time
maintenance_schedule	Lakebase	Planned maintenance windows and crew availability	shift_label, maintenance_window_start, crew_available

Table Name	Schema	Purpose	Key Columns
asset_feature_vectors	Lakebase	Pre-computed feature vectors for fast model serving	equipment_id, feature_vector, feature_names, computed_at

3. Machine Learning Components

> MLflow-managed model lifecycle with per-asset anomaly detection (Isolation Forest) and remaining useful life (RUL) regression models.

ML Training & Scoring Modules

Module	File Path	Purpose	Key Functions
Anomaly Model	`core/ml/anomaly_model.py`	Isolation Forest for unsupervised anomaly detection	`OTPdMAnomalyMode score()`, `log_to_m`
RUL Model	`core/ml/rul_model.py`	Remaining useful life regression	`OTPdMRULModel.fi predict()`, `evalua`

Module	File Path	Purpose	Key Functions
		(Random Forest)	
Training Job	`core/ml/train.py`	Per-industry model training orchestration	`train_asset_mode` experiment trackir
Batch Scoring	`core/ml/batch_score.py`	Batch prediction pipeline (scheduled every 2 hours)	`score_asset()`, ch model resolution, Gold
Feature Engineering	`core/ml/features.py`	Feature matrix construction from Silver tables	`get_feature_matr` pivoting, time-winc aggregation
Feature Importance	`core/ml/feature_importance.py`	SHAP and permutation importance analysis	`compute_feature_` sensor contributio
Model Evaluation	`core/ml/evaluate.py`	RMSE, R^2, MAE metrics for RUL models	`evaluate_rul_mod` precision/recall fo

Anomaly Detection

Algorithm: Isolation Forest (scikit-learn)

Training data: Healthy operation window (first N hours before fault injection)

Output: Anomaly score (0-1), binary label (NORMAL/ANOMALY), top contributing sensor

RUL Prediction

Algorithm: Random Forest Regressor

Label generation: Hours to failure from fault_start_offset

Output: RUL in hours, predicted failure timestamp, confidence interval

MLflow Tracking

Experiment: /Shared/ot-pdm-intelligence/{industry}

Registered models: pdm_{industry}_{equipment_id}_anomaly, _rul

Model registry: Unity Catalog (databricks-uc), Champion/Challenger aliasing

4. Application Backend (FastAPI)

> FastAPI server provides REST APIs for UI, handles SQL orchestration, model payload shaping, and Lakebase operational integration.

Backend Server Module

Component	File Path	Description
FastAPI Server	`app/server.py`	Main application server (231KB, ~6000 lines). Handles all API routes, SQL warehouse queries, currency conversion, Genie integration, and operator action persistence.

Key API Endpoints

Endpoint	Method	Purpose
`/api/overview`	GET	Fleet-level health summary with financial exposure by industry/site
`/api/asset/{id}`	GET	Asset-level telemetry, predictions, and financial context
`/api/geo/sites`	GET	ISA-95 hierarchy navigation with currency-aware rollups
`/api/geo/assets/{site_id}`	GET	Site-level asset list with health and financial metrics
`/api/model/predictions`	GET	Recent ML predictions with model metadata and feature importance
`/api/finance/summary`	GET	Executive finance dashboard with avoided costs and ROI

Endpoint	Method	Purpose
`/api/simulator/inject`	POST	Fault injection for demo scenarios (non-blocking pipeline)
`/api/ui/recommendation/action`	POST	Operator decision capture (Approve/Reject/Defer) to Lakebase
`/api/genie/query`	POST	Natural language query to industry-specific Genie room

Currency & Localization Logic

Backend implements multi-currency conversion with site-native defaults:

- Supported currencies: USD, AUD, JPY, INR, SGD, KRW, AUTO (site-native)
- All financial calculations done in base currency, converted on response
- Language behavior: JPY → Japanese text, KRW → Korean text, others → English
- Geo APIs accept `?currency=XXX` parameter for dynamic conversion

5. Application Frontend (React)

React + Vite frontend with interactive dashboards, ISA-95 hierarchy navigation, factory maps, and Genie integration.

React Components

Component	File Path	Purpose
App.jsx	`app/src/App.jsx`	Main application shell with routing, industry/currency selectors, page navigation
GeoMap.jsx	`app/src/components/GeoMap.jsx`	ISA-95 hierarchy tree navigation with collapsible nodes and health indicators
GeoStatusBar.jsx	`app/src/components/GeoStatusBar.jsx`	Real-time status bar with site filter, risk counts, and financial summary
GeoGeniePanel.jsx	`app/src/components/GeoGeniePanel.jsx`	Natural language query interface with currency-aware prompts and Genie deep-linking
AssetSidebar.jsx	`app/src/components/AssetSidebar.jsx`	Asset detail panel with telemetry charts, predictions, and action buttons

Component	File Path	Purpose
PIDSchematic.jsx	`app/src/components/PIDSchematic.jsx`	P&ID diagram for water/energy industries with live sensor overlays
WindSchematic.jsx	`app/src/components/WindSchematic.jsx`	Wind turbine schematic for energy industry with component health visualization
Sparkline.jsx	`app/src/components/Sparkline.jsx`	Inline time-series charts for trend visualization in tables

App Pages

Fleet Overview

Executive dashboard with portfolio risk matrix, alert list, financial cards, and site-level rollups. Supports currency switching and site filtering.

Asset Drilldown

Asset-specific telemetry, anomaly scores, RUL trends, maintenance recommendations, and historical patterns.

ISA-95 Hierarchy (Geo Map)

Tree-based navigation from site → area → unit → equipment → component. Factory map mode with textured layout and status pins.

Stream Monitor

Recent ingest rows from Bronze tables with quality codes, protocol context, and SDT compression metrics.

Model Insights

ML model outputs, feature importance, advanced maintenance panels with prescriptive recommendations.

Simulator Controls

Fault injection controls, non-blocking inject/score pipeline, "Force critical" demo action, and ingestion configuration.

6. Databricks Jobs & Workflows

> Scheduled and on-demand jobs orchestrate ML training, scoring, financial backfill, and OT connectivity.

Job Definitions (databricks.yml)

Job Name	Schedule	Purpose	Entry Point
ot-pdm-training-{industry}	Daily at 02:00 UTC	Train anomaly and RUL models per asset, register to MLflow	`core/ml/train.py`
ot-pdm-scoring-{industry}	Every 2 hours	Batch prediction using champion models, write to Gold tables	`core/ml/batch_score.py`

Job Name	Schedule	Purpose	Entry Point
ot-pdm-zerobus-connector-{industry}	On-demand	Real-time OT/IoT ingestion from Zerobus protocols	`core/zerobus_ingest/run_connector.py`
ot-pdm-financial-backfill-{industry}	Daily at 01:15 UTC	Generate financial history and planning scenarios (730 days)	`core/finance/generate_financials.py`
ot-pdm-workspace-bootstrap	On-demand	Bootstrap all industries: catalogs, schemas, seed data, grants	`RUNME_BOOTSTRAP_ALL.py` (notebook)

DLT Pipeline Resource

Pipeline Name	Mode	Layers	Configuration
ot-pdm-dlt-{industry}	Continuous	Bronze, Silver, Gold	**Target:** bronze **Catalog:** pdm_{industry} **Workers:** 1

Pipeline Name	Mode	Layers	Configuration
			Libraries: bronze.py, silver.py, gold.py

7. Deployment & Configuration

> Infrastructure-as-code using Databricks Asset Bundles with per-industry parameterization.

Core Deployment Files

File	Path	Purpose
databricks.yml	`databricks.yml`	Bundle definition: jobs, pipelines, app, variables, targets (dev/prod)
app.yaml	`app/app.yaml`	Databricks Apps deployment spec: command, env vars, job IDs,

File	Path	Purpose
		warehouse ID
deployment_matrix.yaml	`industries/deployment_matrix.yaml`	Resource consistency matrix across all industries
schema.sql	`core/catalog/schema.sql`	DDL for all Bronze, Silver, Gold, Lakebase tables

Industry Configuration Files

Industry	Config Path	Key Sections
Mining	`industries/mining/config.yaml`	Accounts: Rio Tinto, BHP. Assets: Cat 793F trucks, conveyors. ISA-95 hierarchy. SDT config.
Energy	`industries/energy/config.yaml`	Accounts: Alinta.

Industry	Config Path	Key Sections
		Assets: Wind turbines, BESS, transformers. Fault injection profiles.
Water	`industries/water/config.yaml`	Accounts: Sydney Water. Assets: Centrifugal pumps, valve systems. Leak detection thresholds.
Automotive	`industries/automotive/config.yaml`	Accounts: Toyota. Assets: Hydraulic presses, robotic welders. Production line stop-risk.
Semiconductor	`industries/semiconductor/config.yaml`	Accounts: Renesas. Assets: Etch chambers, lithography

Industry	Config Path	Key Sections
		steppers. Yield protection physics.

Deployment Commands

```
# Validate bundle
databricks bundle validate --target dev -p DEFAULT

# Deploy specific industry
databricks bundle deploy --target dev -p DEFAULT --var industry=mining

# Deploy all industries
databricks bundle deploy --target dev -p DEFAULT

# Run bootstrap notebook (in Databricks workspace)
RUNME_BOOTSTRAP_ALL.py
```

8. Tools & Utilities

> Supporting scripts for bootstrapping, reconciliation, testing, and operational management.

Tool Scripts

Tool	File Path	Purpose
Bootstrap All Industries	`tools/bootstrap_all_industries.py`	Provisions all configured indu catalogs, scher seed data, grar
Reconcile Industry Matrix	`tools/reconcile_industry_matrix.py`	Audits and auto corrects resour against deployment_ma
Setup Genie Rooms	`tools/setup_genie_rooms.py`	Creates industr specific Genie and writes IDs genie_rooms.js
SDT Compression Report	`tools/sdt_compression_report.py`	Analyzes Swing Door Trending compression ra quality impact
Train & Score from Vectors	`tools/train_and_score_from_feature_vectors.py`	Offline ML experimentatio pre-computed f vectors
Zerobus Easy Start	`tools/zerobus_easy_start.py`	Quick-start wiz Zerobus conne configuration

9. Integration Components

> Modules for OT connectivity, simulation, agent-based reasoning, and external system integration.

Zerobus OT Connectivity

Module	File Path	Purpose
Zerobus Connector	`core/zerobus_ingest/connector.py`	First-mile OT connectivity for OPC-UA, Modbus, MQTT protocols
Connector Runner	`core/zerobus_ingest/run_connector.py`	Job entry point for Zerobus ingestion with error handling and monitoring
Defaults Config	`core/zerobus_ingest/defaults.yaml`	Default protocol configurations and retry policies

Simulator Components

Module	File Path	Purpose
Simulation Engine	`core/simulator/engine.py`	Deterministic telemetry generation with

Module	File Path	Purpose
		configurable tick intervals
Physics Models	`core/simulator/physics.py`	Industry-specific physics: bearing wear, thermal dynamics, hydraulic pressure
Semiconductor Physics	`core/simulator/semiconductor_physics.py`	Fab-specific physics: plasma stability, vacuum integrity, etch rate drift
Fault Injection	`core/simulator/fault_injection.py`	Configurable fault scenarios with gradual degradation and cascade failures
SDT Compression	`core/simulator/sdt.py`	Swinging Door Trending algorithm for data compression

Module	File Path	Purpose
		(85-95% reduction)

Agent & Genie Integration

Module	File Path	Purpose
Agent Core	`core/agent/agent.py`	AI agent orchestration with manual retrieval and context management
Agent Personas	`core/agent/personas.py`	Industry-specific agent personalities and prompt templates
Agent Tools	`core/agent/tools.py`	Tool definitions for SQL queries, manual lookup, and equipment diagnostics
Agent Evaluation	`core/agent/evaluate.py`	Accuracy and response quality metrics for agent performance tuning
Genie Setup	`core/genie/setup.py`	Programmatic Genie room creation with industry-specific instructions

Feature Engineering

Module	File Path	Purpose
Feature Engineering	`core/features/engineering.py`	Shared feature computation logic: rolling stats, FFT, percentile bands

Finance & Planning

Module	File Path	Purpose
Financial Generation	`core/finance/generate_financials.py`	Historical financial data synthesis for executive dashboards (730 days)

Configuration Loader

Module	File Path	Purpose
Config Loader	`core/config/loader.py`	Centralized YAML config loading with inheritance and validation

Summary: Complete Asset Count

3
DLT Pipeline Modules

12
Delta Lake Tables

7
ML Training/Scoring Modules

1
FastAPI Backend Server

9
React Components

5
Scheduled Jobs

5
Simulator Modules

4
Agent/Genie Modules

3
Zerobus Integration

6
Deployment Tools

5

20

Industry Config Files

APJ Sites Configured

For More Information

This asset inventory is current as of the documentation generation date. For the latest code structure and component details, refer to the repository README and inline code documentation.

Repository structure: All source code follows a modular architecture with clear separation between DLT pipelines (`core/dlt/`), ML components (`core/ml/`), application layers (`app/`), and deployment configuration (`industries/`, `databricks.yml`).